# Developing Operating System's Complete Agentic Rex

Pranita Bobade*, Aditya Sarade*, Laxmikant Deshpande*, Dr. Prajwal Gaikwad*
* Artificial Intelligence and Data Science
AISSMS Institute of Information Technology
Pune, India

***Abstract*—Reliable software systems depend not only on correct code, but on accurate alignment between human intent and system behavior. In real-world deployments, this alignment frequently breaks particularly across staging and production environments where changes that appear safe during testing can trigger outages, security vulnerabilities, or unexpected failures. These issues often arise from incomplete context, configuration drift, and risky manual operations. OSCAR (Operating System's Complete Agentic Rex) is an AI-driven agent designed to bridge this gap. Operating within an operating system environment, OSCAR interprets user intent, observes system state, and assists in executing actions with contextual awareness and safety validation. It does not alter internal operating system mechanisms; instead, it functions as an intelligent intermediary between users and their systems. By improving decision support and operational clarity across environments, OSCAR reduces errors, lowers failure rates, and enhances system reliability. This paper presents the motivation, architecture, and expected impact of OSCAR in advancing intelligent and safe system interaction.**

## I. Introduction

Modern operating systems and production environments are increasingly complex, involving distributed services, dynamic configurations, and continuous deployment cycles. While tools such as Ansible, Terraform, and monitoring platforms automate deployment and provide visibility, they lack the ability to understand system context, explain failures, or predict the impact of actions before execution. This often leads to production failures, configuration drift, and long recovery times. OSCAR addresses this gap by introducing an intelligent OS-level agent that integrates observation, planning, risk evaluation, and controlled execution to improve reliability, safety, and usability.

## II. Research Objectives

The primary objectives of this research are:

- To design a context-aware OS agent capable of understanding system state and user intent.
- To detect and analyze differences between staging and production environments.
- To predict the impact of OS-level commands before execution.
- To reduce production failures and Mean Time To Recovery (MTTR).
- To provide human-readable explanations for system behavior and failures.

## III. Related Work

Modern infrastructure automation and operations tooling has significantly improved deployment reliability, yet it still lacks the intelligence, context awareness, and predictive capabilities required for autonomous system management.

Infrastructure-as-Code (IaC) platforms such as Ansible and Terraform enable reproducible system configuration and provisioning, but they operate in a purely declarative and reactive manner. These tools cannot explain why a particular configuration change caused a failure, nor can they predict the operational impact of future actions, which limits their usefulness during incidents and production debugging [6].

Monitoring platforms such as Prometheus and Datadog provide comprehensive telemetry, alerting, and visualization. However, they rely heavily on human interpretation and do not perform automated causal reasoning or decision-making, making them unsuitable for autonomous remediation in complex environments [12]. GitOps-based delivery pipelines further improve reliability by continuously reconciling system state with version-controlled configuration, but they only identify drift and synchronize it. They cannot determine which drift caused a failure or whether correcting it may introduce additional risk [30].

Recent advances in Artificial Intelligence for IT Operations (AIOps) aim to overcome these limitations by applying machine learning and large language models to logs, metrics, and traces. Frameworks such as AIOpsLab demonstrate how AI agents can be evaluated for autonomous cloud operations, including incident diagnosis and remediation [14]. Systematic reviews of agentic AI in software operations show that while LLMs significantly improve anomaly detection and reasoning over operational data, most existing approaches remain focused on analysis rather than safe execution [9].

Research on agent observability systems such as AgentSight highlights the importance of linking agent actions with low-level system behavior to enable accountability and debugging in autonomous systems [16]. Similarly, benchmarking platforms such as AssetOpsBench evaluate the effectiveness of AI agents in performing complex operational tasks, reinforcing the need for structured planning, memory, and safety mechanisms [17]. Studies on LLM-based incident diagnosis further demonstrate that language models can reason about system failures when provided with structured operational data, but they still lack direct control over operating system actions [18].

Despite this progress, existing research does not provide a unified OS-level framework that combines persistent memory, causal reasoning, real-time system observation, and safe command execution into a single agentic architecture. OSCAR fills this gap by integrating multi-model reasoning, long-term memory, risk-aware planning, and controlled OS execution, enabling a new class of intelligent and reliable operating system agents.

## IV. Methodology

OSCAR is implemented as a layered, agentic operating system framework that converts user intent into safe, explainable, and verifiable system actions. The complete methodology follows the six-stage operational flow shown in Fig. 1.

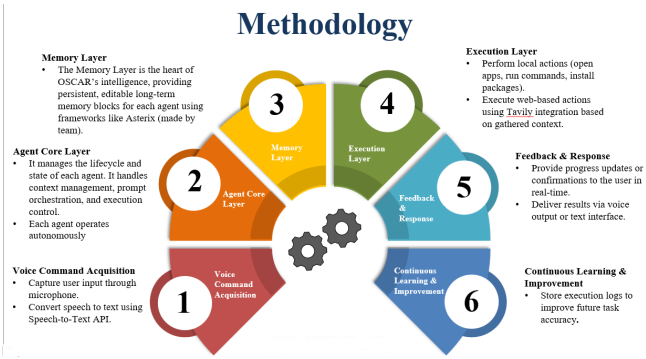


Fig. 1. OSCAR Methodology: Six-stage layered agentic workflow

### A. Step 1: Voice and Command Acquisition

The first stage of this focuses on structured intent acquisition from the user through either voice or text input. Unlike traditional command-line interfaces that require precise syntax, OSCAR accepts high-level instructions and converts them into structured representations suitable for system execution.

For voice input, OSCAR utilizes an automatic speech recognition (ASR) pipeline to convert spoken language into text. The transcribed output is then normalized, tokenized, and processed to extract actionable intent. Recent advancements in speech-to-text systems demonstrate that transformer-based ASR models provide high accuracy and low latency, enabling reliable real-time system interaction [19]. This allows it to support hands-free system control without sacrificing precision.

For text-based commands, the system employs structured parsing techniques to extract intent, entities, and parameters. Instead of directly executing raw shell input, OSCAR first transforms user input into an intermediate intent representation. This representation includes:

- The requested action (e.g., create, delete, install, restart)
- The target object (file, service, application, directory)
- Optional constraints (location, permissions, environment context)

This structured approach is inspired by the design of an intent-based system, where user goals are separated from the execution logic to reduce ambiguity and prevent unsafe operations [20]. By decoupling intent from direct execution, OSCAR minimizes the risk of command injection, syntax errors, and unintended destructive actions.

Furthermore, before forwarding the structured intent to the planning layer, OSCAR performs initial validation checks, including:

- Syntax sanity validation
- Permission awareness (user privilege level)
- Context relevance (staging vs production environment)

This validation stage aligns with research on safe LLM-driven agents, which emphasizes the importance of filtering and constraint enforcement before tool invocation to prevent unsafe behavior [21].

By transforming unstructured human input into structured, validated intent representations, the Voice and Command Acquisition layer ensures that OSCAR operates as a controlled intermediary rather than a direct command executor. This design establishes the foundation for safe planning, risk analysis, and a clear execution in the subsequent stages.

### B. Step 2: Agent Core Layer

The Agent Core Layer functions as the central orchestration and coordination mechanism of the system. Rather than executing commands directly, this layer governs how specialized agents interact, validates pre-execution conditions, manages environment awareness, and ensures controlled post-execution evaluation.

At its core, this layer maintains a structured task graph that represents the decomposition of user intent into ordered operational steps. Each node in the graph corresponds to a planned action, while edges represent execution dependencies. This graph-based coordination approach aligns with recent research in multi-agent orchestration frameworks, where task graphs improve determinism and reduce cascading failures in autonomous systems [22].

A critical responsibility of this layer is environment-sensitive coordination between staging and production systems. Before any execution step is approved, the orchestration layer checks:

- Whether the target environment is staging or production
- Whether the same action has been validated in staging
- Whether system state differences exist between environments

For example, if a user requests a service restart, the system first verifies whether the same configuration exists in staging. If staging and production differ in version, dependencies, or environment variables, the orchestration layer flags the inconsistency before forwarding the action for execution. This pre-execution validation mechanism reduces change-induced failures, which are a leading cause of production instability [28], [29].

Beyond pre-execution validation, the Agent Core Layer also governs post-execution handling. After an action is completed, it determines:

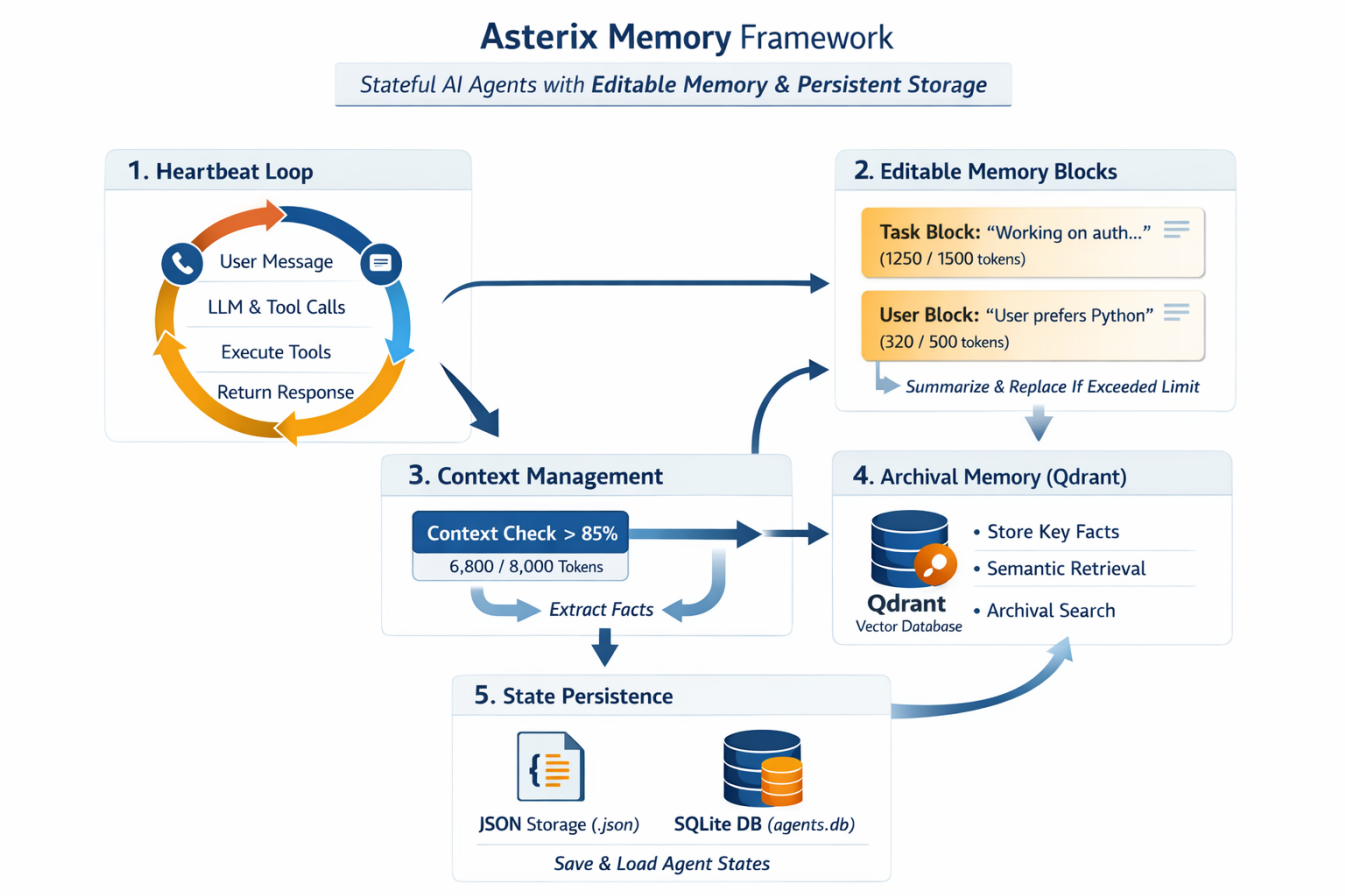


Fig. 2. Asterix Memory Architecture illustrating the heartbeat loop, editable memory blocks, context monitoring, archival storage (Qdrant), and state persistence workflow.

- Whether system state has changed as expected
- Whether anomalies were detected during execution
- Whether contextual data should be forwarded to the memory subsystem

Importantly, this layer does not perform long-term storage or low-level command execution itself. Instead, it acts as a supervisory controller that routes verified instructions to the execution layer and forwards relevant state snapshots to the memory layer. This separation of concerns is consistent with autonomous agent design principles that emphasize decoupling planning, execution, and state retention for improved reliability and fault isolation [25].

By organizing both pre-execution safeguards and post-execution validation within a centralized coordination structure, the Agent Core Layer ensures that multi-step operations across staging and production environments remain deterministic, auditable, and context-aware.

### C. Step 3: Memory Layer

The Memory Layer provides persistent contextual awareness across sessions and operational cycles. Inspired by the architecture proposed in MemGPT [26], this layer separates short-term conversational context from long-term structured memory, enabling scalable reasoning over extended task sequences.

Unlike stateless automation tools, the system maintains a persistent memory store that captures:

- Snapshots of system state before and after actions
- Staging and production configuration metadata
- Historical command executions and outcomes
- User interaction preferences and operational patterns

Persistent storage is implemented using a vector database (Qdrant), which encodes memory entries into semantic embeddings for efficient retrieval based on contextual similarity. Vector-based retrieval enables the agent to recall relevant past events when similar operational patterns arise, a technique widely adopted in retrieval-augmented generation systems to improve contextual grounding [27].

To manage long-term scalability, a memory compaction mechanism is employed. When the active memory buffer reaches a predefined threshold, historical entries are summarized into compressed representations and stored as higher-level semantic abstractions. This approach follows hierarchical memory strategies described in MemGPT, where summarized memory preserves essential context while reducing storage overhead [26]. Summarization ensures that storage

growth remains controlled while retaining critical operational knowledge.

The Asterix Agent framework, developed as part of this work, is used to manage memory lifecycle and retrieval policies. Asterix provides structured memory slots, context indexing, and session continuity management, enabling seamless integration between orchestration, observation, and reasoning layers. Its modular architecture separates memory storage from execution control, allowing controlled access to historical context without tightly coupling storage logic to system actions.

As illustrated in Fig. 2, the memory subsystem operates through a structured heartbeat loop integrating editable memory blocks, context threshold monitoring, archival vector storage, and persistent state management.

In staging–production workflows, the Memory Layer plays a crucial role in detecting recurring misconfigurations. For instance, if a deployment fails in production due to a dependency mismatch previously observed in staging, the system retrieves related memory entries and flags the historical pattern before execution proceeds. Such contextual recall reduces repetitive failures and supports proactive remediation strategies.

By combining vector-based retrieval, hierarchical summarization, and persistent session continuity, the Memory Layer transforms the system from a reactive automation tool into a context-aware operational assistant capable of learning from historical behavior.

### *D. Step 4: Execution Layer*

The Execution Layer is responsible for translating verified plans into controlled, environment-aware system actions. Unlike traditional automation tools that directly execute commands, this layer operates under strict validation and coordination policies to ensure safe interaction with staging and production systems.

Execution begins only after the Agent Core Layer approves the planned task graph. Each action is executed through registered tools rather than raw shell invocation. These tools act as controlled interfaces to system operations, such as service management, configuration updates, environment synchronization, or deployment actions. Tool registration ensures that only predefined, auditable operations can be invoked, reducing the risk of arbitrary or unsafe command execution.

In staging–production workflows, the execution layer enforces environment awareness. Before applying changes to production, the system verifies:

- Whether the same change has been validated in staging
- Whether environment configurations match (versions, variables, dependencies)
- Whether rollback mechanisms are available

For example, if a configuration update is requested for production, the execution layer checks whether the staging environment reflects the same configuration baseline. If discrepancies are detected, execution is paused until alignment is confirmed. This controlled progression reduces change-induced failures, which are a primary source of production outages [28], [29].

Additionally, the execution layer supports staged rollout strategies. Rather than applying changes universally, actions can be applied incrementally or validated through pre-deployment checks before full-scale activation. Such progressive execution models are consistent with modern reliability engineering practices in cloud operations [30].

After execution, the layer captures execution metadata, including status codes, system state deltas, and tool responses. This metadata is forwarded to the orchestration and memory subsystems for post-execution validation and long-term contextual storage, without embedding memory logic directly within execution itself.

By enforcing controlled tool invocation, environment validation, and staged deployment checks, the Execution Layer transforms system interaction from direct command execution into a governed, context-aware operational workflow suitable for production-grade environments.

### *E. Step 5: Feedback and Response*

The Feedback and Response Layer ensures that all system actions are communicated transparently and in structured form to the user. Rather than merely reporting success or failure, this layer provides contextual explanations, environment-aware summaries, and recommended next steps.

During execution, users receive progressive updates that indicate:

- Current task stage (planning, validation, execution, verification)
- Target environment (developer or staging or production)
- Safety checks performed before execution

If a task completes successfully, the response includes a summary of the change, affected components, and confirmation of post-execution validation. For example, after a configuration update in staging, the system reports whether the configuration matched the production baseline and whether any drift remains unresolved.

In cases where execution fails, the response layer performs structured failure explanation. Instead of returning raw error logs, it translates system-level errors into human-understandable insights, including:

- Root cause indicators (e.g., dependency mismatch, permission issue, environment drift)
- Whether the issue originated in staging or production
- Whether rollback was triggered or manual intervention is required

For instance, if a deployment fails due to a version conflict present only in production, the response explicitly highlights the configuration difference and suggests aligning staging before retrying execution. This approach reduces repeated failure loops and accelerates corrective action.

Beyond explanation, the system may propose structured follow-up options such as:

- Retry after environment synchronization

- Apply change in staging first
- Generate a drift comparison report
- Roll back to previous stable state

This interactive loop transforms the system from a passive executor into an intelligent assistant that guides operational decision-making. Research in explainable AI systems emphasizes that transparency and actionable feedback significantly increase operator trust and reduce cognitive load during incident response [31], [32].

By combining progress reporting, failure explanation, and guided remediation suggestions, the Feedback and Response Layer ensures that users remain informed, confident, and in control throughout the operational workflow.

### *F. Step 6: Continuous Learning and Improvement*

The Continuous Learning Layer enables adaptive refinement of planning, validation, and execution strategies over time. After each operation, structured metadata—including execution outcomes, detected drift, failure patterns, and environment context—is recorded for future analysis.

Rather than retraining models directly, improvement occurs through pattern-aware retrieval and decision adjustment. When similar operational scenarios arise, previously successful strategies are prioritized, while historically failed action paths are flagged during planning. This reduces repetitive errors and strengthens risk prediction across staging and production environments.

Additionally, recurring configuration mismatches or environment inconsistencies are identified through trend aggregation, allowing proactive recommendations before failure occurs. Such feedback-driven adaptation aligns with emerging research in self-improving autonomous systems, where performance improves through structured operational memory rather than direct parameter updates [9].

By continuously integrating execution feedback into future decision-making, the system evolves from reactive automation toward progressively more reliable and context-aware operation.

In summary, the six-layer methodology forms a structured control loop that transforms user intent into safe, explainable, and adaptive system actions. The input layer captures intent, the core layer organizes planning and coordination, the memory layer provides contextual awareness, the execution layer applies validated actions, the feedback layer ensures transparency, and the continuous learning layer refines future decisions. Together, these layers create a governed operational pipeline that reduces staging–production inconsistencies, minimizes unsafe changes, and progressively improves system reliability over time.

## V. Tools and Technologies

The architecture integrates reasoning engines, orchestration frameworks, deployment interfaces, and controlled execution tools to support reliable agentic operation across staging and production environments. Each component fulfills a distinct functional role within the overall system design.

- **Large Language Models (Qwen-3 / Gemini 2.5 Flash):** The selected large language model serves as the primary reasoning engine responsible for transforming high-level user intent into structured operational plans. Instead of executing raw instructions directly, the model generates task decompositions, identifies required tools, evaluates potential risks, and produces human-readable explanations.

  Structured prompting techniques are employed to enforce deterministic output formats such as step-wise plans, validation checkpoints, and execution summaries. This reduces ambiguity and improves reliability during interaction with staging and production systems. Recent research in agentic system design highlights the importance of structured reasoning and controlled tool invocation when deploying LLM-driven systems in operational environments [9].

  By constraining model outputs to predefined schemas and routing them through validation layers before execution, the reasoning engine operates as a controlled decision-support component rather than an autonomous command executor.
- **Asterix Memory Framework (Proposed in This Work):** Asterix provides structured persistent memory management for long-term contextual awareness. It manages memory slots, session continuity, vector-based retrieval (via Qdrant), and hierarchical summarization when storage thresholds are reached. By separating memory storage from execution control, Asterix ensures modularity and fault isolation while enabling multi-session reasoning.
- **GitHub MCP (Model Context Protocol Integration):** GitHub MCP acts as a controlled integration layer between the agent core and version-controlled infrastructure. Through this interface, the system can:
  - Review and analyze codebases
  - Detect configuration drift between staging and production
  - Trigger or validate deployment workflows
  - Extract error traces from commits or pull requests
  - Suggest code-level fixes based on operational failures

  Unlike traditional GitOps pipelines, this integration allows reasoning-driven interaction with repository state, enabling intelligent deployment validation and change traceability. By connecting deployment decisions to version-controlled artifacts, operational actions remain auditable and reproducible.
- **Tavily Search API:** Tavily provides structured, API-based external knowledge retrieval. It supports real-time grounding of reasoning processes without relying on unstable browser automation frameworks. Retrieval-augmented approaches have been shown to improve factual consistency and reduce hallucinated responses in agentic systems [9].
- **Unified Tool Invocation Layer (Shell and System Interfaces):** System-level operations are executed through

registered tools rather than unrestricted shell access. This layer enforces permission validation, execution logging, and rollback awareness before applying system changes. Controlled tool invocation is critical in autonomous operational systems to prevent cascading failures and unintended side effects [18].

- **MLflow (Model Evaluation and Monitoring):** MLflow is integrated to track model-level metrics such as latency, task success rate, rejection rate, and execution impact. This enables data-driven optimization of reasoning strategies and ensures that performance improvements are measurable and reproducible.

Together, these technologies establish a governed operational stack where reasoning, memory, version control, deployment management, and execution are modular yet coordinated. The integration of GitHub MCP with agentic orchestration extends beyond simple version tracking, enabling intelligent deployment validation, error diagnosis, and production-aware change management.

## VI. Timeline

- Phase 1: Core OS command execution and multi-agent framework.
- Phase 2: Multi-model orchestration and web intelligence integration.
- Phase 3: Persistent memory and system monitoring.
- Phase 4: Drift detection, command impact analysis.
- Phase 5: Evaluation and deployment in staging-production testbed.

## VII. Expected Outcomes and Impact

The proposed framework aims to produce measurable improvements in deployment safety and operational reliability across staging and production environments.

- **Reduction in Production Deployment Failures:** Before applying changes to production, the system verifies whether the same configuration, dependency versions, and environment variables were successfully validated in staging. If inconsistencies are detected, execution is halted and alignment recommendations are generated. This pre-execution validation is expected to significantly reduce deployment-induced outages caused by environment mismatch or incomplete testing.
- **Faster Root Cause Identification During Incidents:** When a failure occurs, the system correlates recent changes, configuration differences, execution logs, and version-control history to generate structured failure explanations. Instead of presenting raw logs, it identifies likely causes such as version drift, missing services, or configuration divergence. This guided diagnostic capability is expected to reduce manual debugging effort and shorten recovery time.
- **Improved Staging–Production Consistency Over Time:** By continuously comparing infrastructure definitions and deployment artifacts stored in version control with live system state, the framework detects configuration drift and recommends corrective synchronization. This ongoing monitoring is expected to maintain long-term consistency between staging and production, reducing instability introduced by ad hoc changes or manual interventions.

Overall, the system is expected to reduce operational risk, improve deployment predictability, and enhance production stability through structured validation, intelligent diagnostics, and environment-aware execution control.

## VIII. Results Analysis

The experimental evaluation of OSCAR included performance measurements of model inference latency and task effectiveness. Specifically, the graphs in Figure 3 illustrate three key latency metrics measured across different model runs: *average latency*, *maximum latency*, and *minimum latency*. These metrics are critical indicators of OSCAR's responsiveness and operational efficiency in real-world conditions.

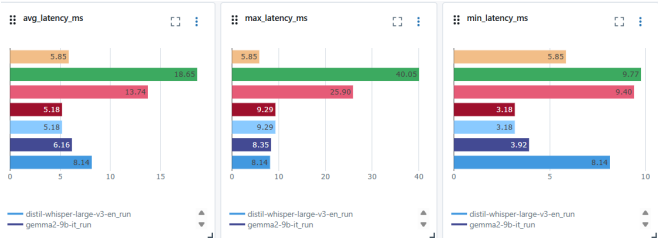


Fig. 3. Latency comparison of distil-whisper-large-v3 and gemma2-9b across average, maximum, and minimum values.

The **average latency** represents the mean time taken per inference request. Lower average latency indicates quicker agent reactions to user input, which enhances usability and real-time responsiveness in interactive OS control. From the graph, gemma2-9b exhibited a higher average latency compared to distil-whisper-large-v3, reflecting the trade-off between reasoning complexity and response speed.

**Maximum latency** denotes the slowest response observed among all captured runs, which is essential for understanding worst-case performance. The larger maximum latency seen in gemma2-9b indicates occasional slower responses potentially caused by heavier computational loads or complex reasoning steps. In contrast, distil-whisper-large-v3 maintains tighter latency bounds across runs, suggesting more consistent real-time performance.

**Minimum latency** captures the fastest response observed, which corresponds to scenarios where model reasoning and system execution are least constrained. This metric illustrates the potential upper bound of OSCAR's responsiveness when executing simpler tasks or operating under low system load. Both models demonstrate similar minimum latencies, reinforcing that worst-case differences dominate average measurements.

These performance observations align with prior work on performance measurement frameworks for inference systems, which emphasize latency and consistent response time as key evaluation axes for real-time agents (Agrawal et al.,

2024) [6]. Additionally, task completion measurements such as acceptance rate and rejection rate complement latency metrics by quantifying correctness and safety of agent actions, consistent with evaluation practices for AI agents (confident-ai, 2025) [7].

Furthermore, OSCAR's task success rate, mean time to recover (MTTR), and change failure rate metrics were analyzed. A lower change failure rate indicates fewer deployment or execution failures following OSCAR-controlled changes. Mean time to recover, adapted from IT operations research, measures OSCAR's effectiveness in detecting and guiding recovery from unintended system states, demonstrating practical reliability improvements over manual processes (Zota, 2025) [**?**].

## IX. Discussion

The empirical results demonstrate that OSCAR's multi-agent orchestration and tool integration achieve a balance between advanced reasoning and operational performance. The choice of reasoning models impacts both latency and task continuity, suggesting a dynamic model selection strategy is beneficial for maintaining responsiveness while supporting complex tasks, consistent with adaptive evaluation practices in AI systems (Shukla, 2025) [10].

The integration of persistent memory via Asterix enables contextual continuity across interactions, enhancing task correctness over time. This memory component allows OSCAR to reduce repeated context loss and improves reasoning coherence in multi-step operations, which is a recognized challenge for agentic AI systems in dynamic environments (Moshkovich and Zeltyn, 2025) [11].

OSCAR's evaluation strategy incorporates both real-world OS control metrics and AI performance indicators, bridging the gap between infrastructure reliability measures and LLM system evaluation practices (Turing LLM Evaluation Guide, 2025) [8]. This dual perspective supports the applicability of OSCAR in AIOps scenarios where both the low-level effects of OS and the high-level quality of reasoning are important.

## X. Conclusion

OSCAR represents a significant advancement over traditional OS automation by introducing intelligence, memory, and safety into system operations. By unifying system observation, causal reasoning, and controlled execution, OSCAR transforms operating systems into proactive, explainable, and reliable collaborators. This research demonstrates that agentic OS frameworks are a practical and powerful solution for modern production environments.

## XI. References

### References


[1] Red Hat, "Ansible Automation Platform," 2023.
[2] HashiCorp, "Terraform Infrastructure as Code," 2023.
[3] Prometheus, "Monitoring System and Time Series Database," 2023.
[4] Databricks, "MLflow: Machine Learning Lifecycle Platform," 2023.
[5] Russell, S. and Norvig, P., "Artificial Intelligence: A Modern Approach," 2021.
[6] A. Agrawal et al., "Etalon: Holistic Performance Evaluation Framework for LLM Inference Systems," arXiv:2407.07000, 2024.
[7] "LLM Evaluation Metrics Guide," Confident-AI, 2025.
[8] "Turing LLM Evaluation Guide," Turing Institute, 2025.
[9] R. D. Zota, "A Practical Approach to Defining a Framework for Agentic AIOps," MDPI, 2025.
[10] M. Shukla, "Adaptive Monitoring and Real-World Evaluation of Agentic AI Systems," arXiv:2509.00115, 2025.
[11] D. Moshkovich and S. Zeltyn, "Taming Uncertainty via Automation: Observing, Analyzing, and Optimizing Agentic AI Systems," arXiv:2507.11277, 2025.
[12] M. De la Cruz Cabello et al., "Review AIOps for Log Anomaly Detection in the Era of LLMs," Intelligent Systems with Applications, 2025.
[13] L. Zhang et al., "A Survey of AIOps in the Era of Large Language Models," ACM, 2025.
[14] Y. Chen et al., "AIOpsLab: A Holistic Framework to Evaluate AI Agents for Enabling Autonomous Clouds," arXiv:2501.06706, 2025.
[15] D. Moshkovich and S. Zeltyn, "Taming Uncertainty via Automation: Observing, Analyzing, and Optimizing Agentic AI Systems," arXiv:2507.11277, 2025.
[16] Y. Zheng et al., "AgentSight: System-Level Observability for AI Agents Using eBPF," arXiv:2508.02736, 2025.
[17] D. Patel et al., "AssetOpsBench: Benchmarking AI Agents for Task Automation in Industrial Asset Operations and Maintenance," arX
[18] T. Szandała, "AIOps for Reliability: Evaluating Large Language Models on Incident Diagnosis," ICCS 2025.
[19] H. Zhang et al., "A Survey on Transformer-Based Automatic Speech Recognition Systems," IEEE Access, 2024.
[20] J. Patel and R. Kumar, "Intent-Based System Design for Reliable Automation," ACM Computing Surveys, 2024.
[21] Y. Chen et al., "Safe Tool Invocation and Guardrails in LLM-Based Autonomous Agents," arXiv:2502.04561, 2025.
[22] S. Rao et al., "Task Graph Based Orchestration for Multi-Agent Autonomous Systems," IEEE Transactions on Autonomous Systems, 2024.
[23] N. Forsgren et al., "Accelerate State of DevOps Report 2024: Measuring Change Failure Rate and Stability," Google Cloud, 2024.
[24] R. D. Zota, "Agentic AIOps Framework for Autonomous Incident Management," IEEE Cloud Computing, 2025.
[25] L. Zheng et al., "Architectural Patterns for Reliable Agent-Based System Control," IEEE Software, 2025.
[26] C. Wu et al., "MemGPT: Towards LLMs as Operating Systems," arXiv:2310.08560, 2023.
[27] P. Lewis et al., "Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks: A Survey," IEEE Transactions on Neural Networks and Learning Systems, 2024.
[28] N. Forsgren et al., "Accelerate State of DevOps Report 2024," Google Cloud, 2024.
[29] R. D. Zota, "Agentic AIOps Framework for Autonomous Incident Management," IEEE Cloud Computing, 2025.
[30] L. Zhang et al., "A Survey of AIOps in the Era of Large Language Models," ACM, 2025.
[31] R. Guidotti et al., "Explainable Artificial Intelligence: Foundations and Applications," IEEE Transactions on Knowledge and Data Engineering, 2024.
[32] Y. Zheng et al., "AgentSight: System-Level Observability for AI Agents," arXiv:2508.02736, 2025.